Семинар 3

Цель работы: разработка объектно-ориентированной модели банковской системы с использованием языка C++, включающей:

- работу с классами и объектами
- реализацию наследования
- организацию взаимодействия между сущностями
- моделирование банковских операций

Задачи: необходимо разработать систему, моделирующую работу банка, которая должна:

- хранить информацию о клиентах
- поддерживать различные типы счетов
- выполнять операции (пополнение, снятие, перевод)
- хранить историю операций
- выводить информацию в удобном виде

2. Описание предметной области

Предметной областью данной работы является деятельность банковских организаций, связанная с обслуживанием клиентов и управлением их финансовыми средствами. Банк предоставляет клиентам возможность открытия различных типов счетов, проведения финансовых операций и хранения информации о движении денежных средств.

2.1 Банк и его функции

Банк представляет собой организацию, которая регистрирует клиентов, открывает и обслуживает счета, выполняет финансовые операции и рассчитывает комиссии.

2.2 Клиенты банка

Клиент — это физическое лицо, зарегистрированное в банке, которое может владеть одним или несколькими счетами и выполнять операции со своими средствами.

2.3 Банковские счета

Счет является основной сущностью системы. Он содержит номер, владельца, баланс и историю операций. В системе реализованы сберегательные, кредитные и депозитные счета.

2.4 Финансовые операции

Поддерживаются операции пополнения, снятия, перевода, оплаты картой, начисления процентов и списания комиссий.

2.5 История операций

Для каждого счета ведется история операций, включающая тип операции, сумму и комментарий.

2.6 Банковские карты

Карта связана со счетом и позволяет выполнять оплату и снятие средств.

2.7 Комиссии

Расчет комиссии выполняется отдельным компонентом системы, что позволяет централизовать логику расчетов.

3. Архитектура системы

Система построена на основе ООП и включает следующие сущности:

Основные классы:

Bank	хранит список клиентов и отделений
	выполняет переводы между счетами

Client	содержит информацию о клиенте
	хранит список счетов

Account (базовый класс)	общий интерфейс для счетов
	1. номер счета 2. владельца 3. баланс 4. историю операций

Производные классы счетов:

SavingsAccount	начисляет проценты

CreditAccount	поддерживает кредитный лимит

Дополнительные классы:

Card	привязана к счету
	позволяет оплачивать покупки

Transaction	тип операции
	сумму + комментарий

Department	отделение банка

4. Взаимодействие классов

Взаимодействие классов представлено на иллюстрации ниже

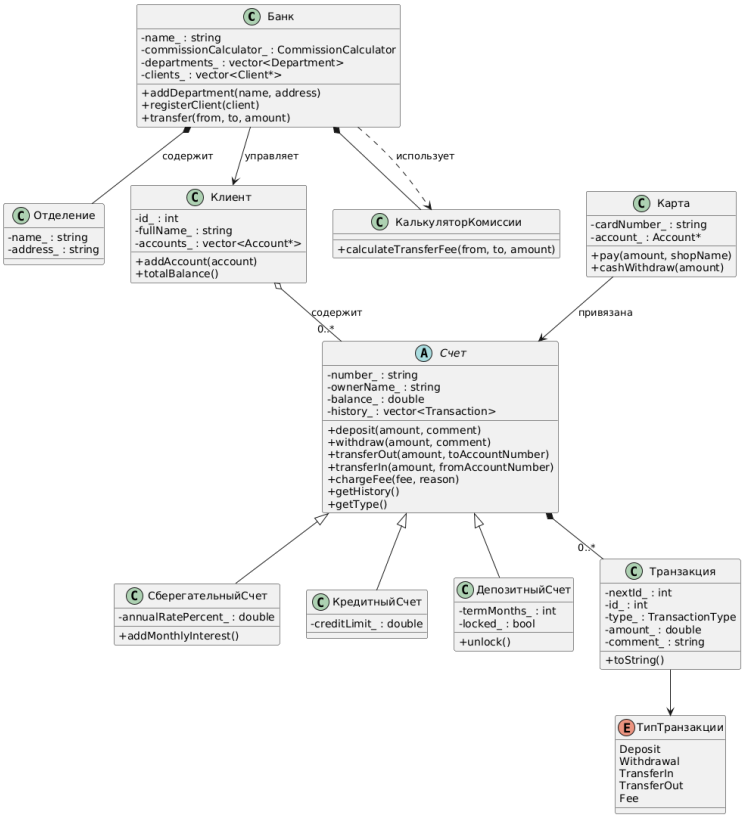

Рисунок 1 – Диаграмма классов банковской системы

На диаграмме показаны основные типы связей:

- наследование

- агрегация

- композиция

- ассоциация

На рисунке 2 показаны связи классов:

Где показаны связи

- **Наследование**: `Account -> SavingsAccount, CreditAccount, DepositAccount`.
- **Ассоциация**: `Card` хранит `Account*`.
- **Агрегация**: `Client` хранит `std::vector<Account*>`.
- **Композиция**: `Bank` содержит `Department` и `CommissionCalculator`; `Account` содержит `Transaction`.
- **Делегирование**: `Bank::transfer` делегирует расчет комиссии `CommissionCalculator`.

Рисунок 2 – Связь классов

Диаграмма отражает структуру системы, а также типы связей между классами, включая наследование, агрегацию, композицию и ассоциацию.

5. Реализация системы

Система реализована на языке программирования C++ с использованием принципов объектно-ориентированного программирования.

Для каждого класса создана отдельная пара файлов (.h и .cpp), что обеспечивает модульность и удобство сопровождения кода.

Основные элементы реализации:

- Класс Bank реализует управление клиентами и выполнение переводов

- Класс Client хранит счета клиента и вычисляет суммарный баланс

- Базовый класс Account содержит общую логику работы со счетами

- Производные классы реализуют специализированное поведение

- Класс Transaction отвечает за хранение истории операций

- Класс Card реализует работу с банковской картой

- Класс CommissionCalculator выполняет расчет комиссии

6. Демонстрация работы программы

В ходе выполнения программы выполняются следующие действия:

1. Создание банка и отделений

2. Регистрация клиентов

3. Создание различных типов счетов

4. Выполнение операций:

- пополнение счета
- снятие средств
- перевод между счетами
- оплата картой
- начисление процентов

5. Вывод итогового состояния счетов

6. Вывод истории операций

Программа выводит результаты в консоль в структурированном и читаемом виде.

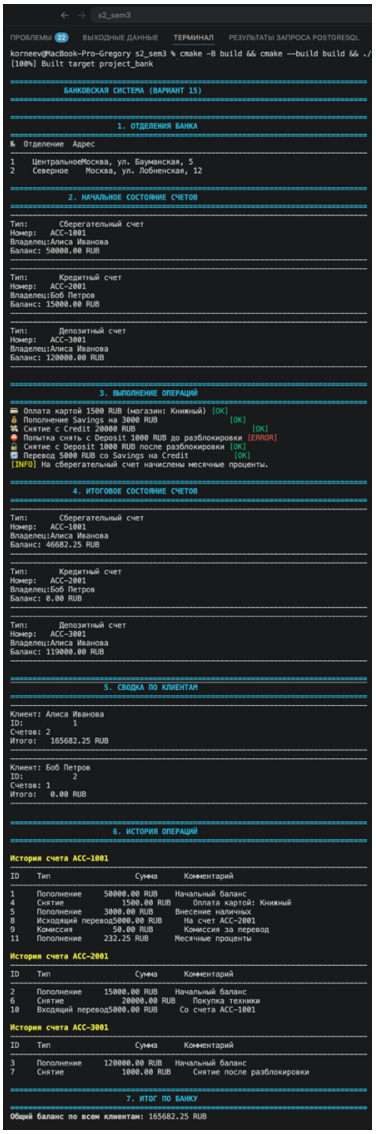

Рисунок 3 – Демонстрация работы в терминале VS Code

7. Вывод

В ходе выполнения работы была разработана объектно-ориентированная модель банковской системы.

В результате выполнения программы:

- *реализовано создание клиентов и различных типов счетов*

- *обеспечена корректная работа операций пополнения, снятия и перевода средств*

- *реализована поддержка банковских карт для оплаты*

- *организовано хранение и вывод истории операций*

- *реализован механизм расчета комиссии при переводах*

- *обеспечен наглядный и структурированный вывод результатов в консоль*

Разработанная система успешно выполняет поставленные задачи, демонстрирует корректную работу всех компонентов и полностью соответствует техническому заданию.

Работа демонстрирует практическое применение принципов объектно-ориентированного программирования при разработке программных систем.